

Projekt WDUM Raport

Mateusz Brochocki
Nr indeksu 327177

1 Wstęp

Celem naszej pracy jest stworzenie modelu, który najlepiej przewidzi ryzyko kredytowe klientów pewnego banku i zakwalifikuje ich do jednej z dwóch grup *Good* albo *Bad*. Będziemy starali się stworzyć taki model klasyfikacji, który zapewni największą wartość miary *balanced accuracy*.

2 Zbiór danych i jego przygotowanie

Po wstępnej eksploracji możemy zauważyć, że nasz zbiór X_{train} składa się 3360 wierszy i 24 kolumn, przy czym pierwszą kolumnę usuwamy od razu, gdyż jest to numer charakterystyczny dla każdego klienta i nie wnosi on nic do dalszej predykcji. Zamieniamy także liczby oznaczające braki danych na *NaN* i zauważamy, że jest ich całkiem sporo. Ponadto na podstawie histogramów zauważamy, że występują outliery, a wiele rozkładów jest lewostronnie i prawostronnie skośnych. Rozkład zmiennej celu nie jest zaburzony (mniej więcej po połowie obu kategorii). Mimo, że wszystkie zmienne są oznaczone jako ciągłe, to będziemy traktować X_{10} i X_{11} jako zmienne dyskretnie. Pierwszym krokiem będzie transformacja zmiennych skośnych odpowiednio prawostronnie skośnych funkcją $\ln(1 + X)$, a lewostronnie skośnych $\ln(\frac{1}{1+X} + 1)$. Kolejnym krokiem jest uzupełnienie braków. W przypadku zmiennych kategoriycznych uzupełniamy najczęściej występującą wartością, zaś w przypadku pozostałych zmiennych korzystamy z funkcji *IterativeImputer*, która dobrze się sprawdza w przypadku dużej liczby braków danych, które mogą mieć istotny wpływ na model, a jednocześnie nie chcemy używać metody k-najbliższych sąsiadów, gdyż nasze zmienne nie są jeszcze przeskalowane. Kolejnym krokiem jest właśnie skalowanie zmiennych, które robimy za pomocą *StandardScaler()*. Na końcu dokonujemy zaawansowanej selekcji zmiennych za pomocą funkcji *SelectFromModel*, w której rolę selektora pełni model regresji logistycznej z regularyzacją l_2 . Sprawdza się on nam w tym przypadku, gdyż mamy dużo zmiennych o bardzo różnych zależnościach między sobą. Pozwala nam to na wyselekcjonowanie 8 najistotniejszych zmiennych. Dzielimy zbiór X_{train} na zbiór treningowy i testowy w stosunku 85 : 15.

3 Budowa modeli prostych

Będziemy sprawdzać, jak zachowują się podstawowe modele na naszych danych, a jednocześnie postaramy się dobrać dla nich optymalne hiperparametry używając za każdym razem potrójnej krosvalidacji i przeszukując siatkę ze z góry ustalonych parametrami.

3.1 Drzewo decyzyjne

Poszukujemy tutaj czterech optymalnych parametrów modelu: *criterion*, *max_depth*, *min_samples_leaf* i *min_samples_split*, oznaczających kolejno: kryterium podziału, maksymalną głębokość drzewa, minimalną liczbę rekordów w liściu i minimalną wymaganą liczbę rekordów do podziału liścia. Hiperparametr *criterion* będzie miał potencjalne wartości *gini* i *entropy*, *max_depth* liczby całkowite z przedziału od 3 do 10, *min_samples_leaf* liczby całkowite z przedziału od 5 do 100 co 5, a *min_samples_split* liczby całkowite z przedziału od 5 do 250 co 5. Okazuje się, że optymalnymi parametrami są *criterion = entropy*, *max_depth = 4*, *min_samples_leaf = 40*, *min_samples_split = 195*.

3.2 Las losowy

W tym przypadku szukamy jednego parametru - *n_estimators* oznaczającego liczbę drzew użytych w modelu. Poszukujemy jego wartości wśród liczb całkowitych od 50 do 600 co 50. Nie poszukujemy innych hiperparametrów, gdyż trwałoby to zbyt długo, ale korzystamy z wyników uzyskanych w poprzednim eksperymencie dla pojedynczego drzewa. Ustawienie ich zapobiegnie przeuczeniu się lasu. Optymalna wartość parametru to *n_estimators = 50*.

3.3 ExtraTrees

W tym modelu mamy sytuację anlogiczną do powyższej. Jediną różnicą jest wynik, gdyż tym razem *n_estimators = 100*.

3.4 Regesja logistyczna z regularyzacją *l1*

Szukanyimi parametrami są hiperparametry: *solver*, *C*, *class_weight*, które odpowiadają kolejno za: metodę rozwiązywania zadania optymalizacji odpowiedniej funkcji, siłę regularyzacji, która w tym przypadku związana jest z metryką *l1* oraz fakt czy odpowiednie klasy będą miały wagi. Siatka hiperparametrów składa się z: typu solvera dostępnego dla naszej regularyzacji, wartości parametru *C* - 0.001, 0.01, 0.1, 0.25, 0.5, 0.75, 1, 5, 10, 100 - oraz opcji *None* i *balanced* dla *class_weight*. Optymalne wartości to *solver = saga*, *C = 0.25*, *class_weight = None*.

3.5 Regesja logistyczna z regularyzacją *l2*

W tym przypadku sytuacja jest analogiczna do powyższej z tą różnicą, że stosujemy regularyzację *l2* i dzięki temu mamy dostęp do większej liczby solverów. Tutaj najlepszymi hiperparametrami są *solver = saga*, *C = 0.1*, *class_weight = None*.

3.6 SVC

W modelu wektorów podpierających poszukiwanymi parametrami są *kernel* i *C*, które odpowiadają odpowiednio za sposób rozwiązania problemu podziału podprzestrzeni na dwie klasy i siłę regularyzacji. Możliwe wartości to *linear*, *poly*, *rbf*, *sigmoid* dla parametru *kernel*, zaś dla parametru *C* takie same jak w przypadku regresji. Optymalnymi hiperparametrami są *kernel = rbf* oraz *C = 0.5*.

3.7 LDA

Kolejny model związany jest z liniową analizą dyskryminacyjną. Optymalizowanym parametrem będzie *solver* z możliwymi wartościami *svd*, *lsqr* oraz *eigen*. Najlepszą jakość daje ustawienie hiperparametru *solver* = *svd*.

3.8 QDA

W tym modelu zajmujemy się kwadratową analizą dyskryminacyjną. Optymalizować będziemy parametr *reg_param* odpowiadający za regularyzację macierzy kowariancji. Ten parametr może przyjmować wartości od 0 do 1, a zatem nasza siatka będzie składać się z wartości od 0 do 1 co 0.1. Optymalnie *reg_param* = 0.3.

3.9 k-najbliższych sąsiadów

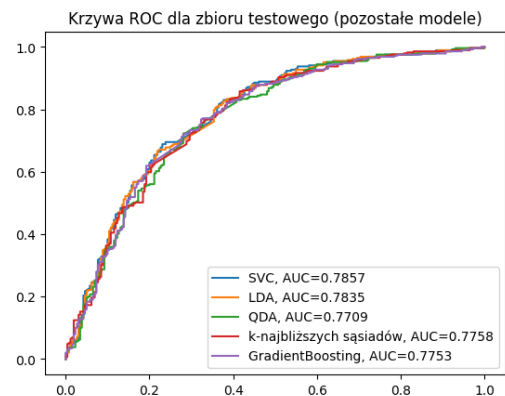
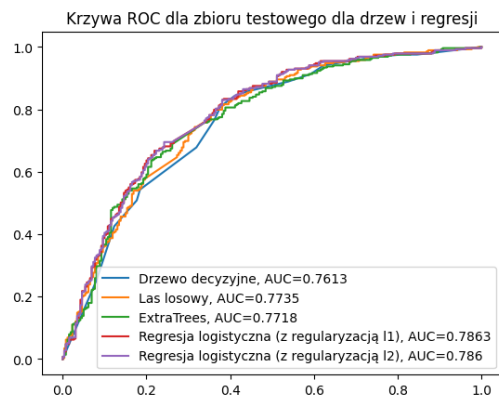
Tym razem optymalizujemy trzy parametry: *n_neighbors*, *weights* oraz *p*. Odpowiadają one kolejno za: liczbę uwzględnianych sąsiadów, doważanie sąsiadów (im dalej tym mniejszy wpływ), oraz normę, w której mierzona jest odległość między rekordami. Nasza siatka parametrów zbudowana jest z liczb całkowitych od 5 do 200 co 5 dla hiperparametru *n_neighbors*, argumentów *uniform* i *distance* dla hiperparametru *weights* oraz wartości 1, 2 i 3 dla hiperparametru *p*. Optymalne wartości to *n_neighbors* = 40, *weights* = *distance* oraz *p* = 2.

3.10 GradientBoosting

W ostatnim modelu optymalizacji podlegają trzy parametry: *loss*, *learning_rate*, *n_estimators*. Odpowiadające za funkcję straty, wkład poszczególnego modelu i ich liczbę. Potencjalne wartości hiperparametrów to: *log_loss* i *exponential* dla parametru *loss*, liczby 0.001, 0.01, 0.1, 0.25, 0.5, 0.75, 1, 5, 10, 100, 1000 dla parametru *learning_rate* oraz liczby 1, 25, 50, 75, 100, 150, 200, 500, 1000 dla parametru *n_estimators*. Okazuje się, że optymalnymi wartościami parametrów są *loss* = *exponential*, *learning_rate* = 0.01 oraz *n_estimators* = 500.

4 Wyniki i wnioski dla modeli prostych

Model	Balanced accuracy - trening	Balanced accuracy - test
Drzewo decyzyjne	0.7227	0.7079
Las losowy	0.7343	0.7145
ExtraTrees	0.7153	0.6923
Regresja logistyczna (regularyzacja l1)	0.7321	0.7181
Regresja logistyczna (regularyzacja l2)	0.7305	0.7164
Wektory podpierające	0.7383	0.7268
LDA	0.7275	0.7218
QDA	0.7129	0.7060
k-najbliższych sąsiadów	0.9793	0.7125
GradientBoosting	0.7531	0.7125



Widzimy, że tak naprawdę wszystkie modele mają porównywalną jakość. Najlepszy okazał się model zbudowany na wektorach podpierających, a najslabszy model *ExtraTrees*. Wszystkie modele poza modelem k-najbliższych sąsiadów nie wykazują przejawów przeczenia. Wykazują się one satysfakcjonującą jakością z AUC na poziomie ok. 0.78 i *balanced_accuracy* ok. 0.715 na zbiorze testowym, którą jednak spróbujemy poprawić modelami złożonymi.

5 Modele złożone

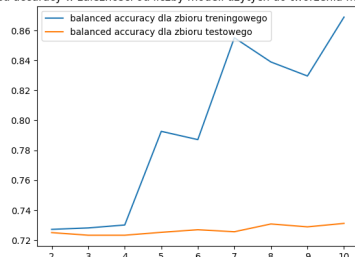
5.1 Bagging

Pierwszym modelem złożonym będzie *bagging* zbudowanym na najlepszym modelu, czyli SVM. Jednocześnie będziemy próbowali zoptymalizować w nim parametr *n_estimators* odpowiedzialny za liczbę próbek. Okazuje się, że optymalną wartością z zadanej przez nas siatki (25, 50, 100, 150, 200, 250, 300) jest *n_estimators* = 200. Jednak model na zbiorze testowym osiągnął dokładnie taką samą miarę *balanced_accuracy* jak pojedynczy SVM.

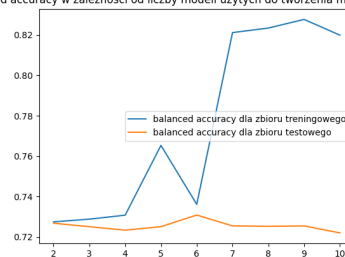
5.2 Voting i stacking

W przypadku tych dwóch modeli przeprowadzimy eksperyment, ile najlepszych modeli prostych, biorąc pod uwagę *balanced_accuracy* na zbiorze testowym, powinniśmy użyć, aby uzyskać najlepszy wynik.

Miara *balanced accuracy* w zależności od liczby modeli użytych do tworzenia modelu metodą voting



Miara *balanced accuracy* w zależności od liczby modeli użytych do tworzenia modelu metodą stacking



Widzimy, że w przypadku *votingu* są to wszystkie modele, zaś dla *stackingu* jest to 6 modeli. W tym przypadku modele złożone poprawiły jakość predykcji w stosunku do modeli prostych. Porównując nasze optymalne modele, *voting* wykazuje minimalnie lepszą jakość na zbiorze testowym i znacznie lepszą na zbiorze treningowym, dlatego będzie to nasz optymalny model.